

Curso de C++

Capítulo 2

Entradas de Usuario en C++

Desde principiante hasta nivel práctico

Enfoque: Desarrollo de C++

Objetivo del capítulo

Aprenderás a recibir información escrita por el usuario desde la consola de C++, almacenarla en variables, validarla y utilizarla para producir resultados. El capítulo avanza desde cin y entradas simples hasta getline, errores de formato y un pequeño sistema interactivo.

Al finalizar este capítulo serás capaz de

- Explicar qué es una entrada de usuario y qué problema resuelve.
- Utilizar cin para leer números, caracteres y palabras.
- Utilizar getline para leer líneas completas.
- Distinguir entre captura, validación y procesamiento.
- Detectar fallos básicos con cin.fail().
- Aplicar entradas en inventarios, máquinas, clientes y producción.

Introducción al tema

Una entrada de usuario es información que una persona proporciona mientras el programa está ejecutándose: un nombre, una cantidad, una temperatura, una opción o una descripción. En una aplicación de consola, normalmente se introduce mediante el teclado.

Las entradas existen para que el programa no dependa de valores escritos permanentemente por el programador. Permiten reutilizar el mismo programa con diferentes datos y situaciones. En un sistema real, la información podría venir de un formulario, un archivo, un PLC o un sensor; en este capítulo el teclado representa esa fuente.

La relación con el capítulo anterior es directa: las variables almacenan datos y las entradas permiten llenar esas variables. El flujo general es: **solicitar** → **leer** → **almacenar** → **validar** → **procesar** → **mostrar**.

1. cin: recibir datos

¿Qué es?

cin es un objeto de la biblioteca estándar iostream que recibe datos desde la entrada estándar, normalmente el teclado.

¿Por qué y cuándo se utiliza?

Se utiliza cuando el programa necesita recibir datos durante su ejecución. Es apropiado para enteros, decimales, caracteres y palabras.

Sintaxis

```
cin >> variable;
```

Ejemplo básico

```
#include <iostream>
using namespace std;

int main() {
    int edad;
    cout << "Ingresa su edad: ";
    cin >> edad;
    cout << "Edad registrada: " << edad << endl;
    return 0;
}
```

Línea por línea: `iostream` habilita la entrada y salida; `edad` es una variable entera; `cout` solicita el dato; `cin >> edad` espera la escritura y almacena el valor; la última línea muestra el contenido actualizado. Si se escribe 25, `edad` queda en 25.

2. El operador >>

El operador `>>` extrae información del flujo de entrada y la coloca en una variable. El tipo de la variable determina cómo se interpreta el dato.

```
int piezas;
double temperatura;
char categoria;

cin >> piezas;
cin >> temperatura;
cin >> categoria;
```

La extracción suele ignorar espacios iniciales y termina al encontrar un separador apropiado. Por eso `cin >> nombre` lee una palabra, no necesariamente una frase completa.

Ejemplo intermedio

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string maquina;
    int piezas;
    double temperatura;

    cout << "Maquina: ";
    cin >> maquina;
    cout << "Piezas: ";
    cin >> piezas;
    cout << "Temperatura: ";
    cin >> temperatura;

    cout << maquina << " - " << piezas
    << " piezas - " << temperatura << " C" << endl;
    return 0;
}
```

El programa combina variables del capítulo anterior con entradas. Si se escribe 'CNC Principal', solo se capturará 'CNC'. Para texto con espacios se necesita `getline`.

3. getline: leer una línea completa

¿Qué es?

`getline` es una función que lee caracteres hasta encontrar el salto de línea. Conserva los espacios y es ideal para nombres completos, direcciones y descripciones.

Sintaxis

```
getline(cin, variableTexto);

string nombreCompleto;
cout << "Nombre completo: ";
getline(cin, nombreCompleto);
```

Si el usuario escribe 'Ana López García', `getline` almacena toda la frase. En cambio, `cin >> nombreCompleto` se detendría después de 'Ana'.

Comparación

Método	Entrada	Resultado
cin >>	CNC Principal	CNC
getline	CNC Principal	CNC Principal

4. El salto de línea pendiente

Cuando se usa `cin >>` y después `getline`, el Enter que termina la lectura puede permanecer en el flujo. `getline` puede encontrarlo inmediatamente y leer una línea vacía.

```
int edad;
string nombre;

cin >> edad;
getline(cin, nombre); // Puede leer vacío
```

Una corrección común es descartar el salto antes de `getline`.

```
cin >> edad;
cin.ignore();
getline(cin, nombre);
```

También puede utilizarse `cin.ignore(1000, '\n')` para descartar caracteres hasta el salto de línea. La idea es limpiar la entrada restante antes de leer la línea completa.

5. Entradas de diferentes tipos

- `int`: cantidades, edades y contadores.
- `double`: temperaturas, precios y mediciones con decimales.
- `char`: una opción o carácter individual.
- `bool`: estados lógicos; para comenzar es claro pedir 1 o 0 y convertirlo.
- `string`: nombres, códigos y descripciones.

```
int respuesta;
cout << "Maquina activa? 1=si, 0=no: ";
cin >> respuesta;
bool activa = respuesta == 1;
```

La comparación produce `true` o `false` y permite convertir una respuesta numérica en un estado lógico.

6. Validación básica

¿Qué es validar?

Validar significa comprobar que el dato cumple las reglas del problema. Una cantidad de piezas no debería ser negativa y una opción de menú debe pertenecer a las opciones permitidas.

```
int piezas;
cout << "Piezas producidas: ";
cin >> piezas;

if (piezas < 0) {
    cout << "Cantidad invalida.";
} else {
    cout << "Cantidad aceptada.";
}
```

La captura recibe el dato; if comprueba una regla; la salida informa el resultado. Leer un dato no significa que sea correcto.

7. Errores de formato y cin.fail()

Si el usuario escribe texto cuando se espera un entero, la extracción puede fallar. cin.fail() permite detectar ese estado.

```
int edad;
cin >> edad;

if (cin.fail()) {
    cout << "Entrada invalida." << endl;
    cin.clear();
    cin.ignore(1000, '\n');
}
```

cin.clear() limpia el estado de error y ignore descarta la entrada incorrecta. En programas grandes conviene encapsular este proceso en una función de validación.

Ejemplos progresivos

Nivel 1 — Una entrada

```
double temperatura;
cout << "Temperatura: ";
cin >> temperatura;
cout << "Registrada: " << temperatura << " C";
```

Nivel 2 — Inventario

```
string producto;
int cantidad;
double precio;

cout << "Producto: ";
cin >> producto;
cout << "Cantidad: ";
cin >> cantidad;
cout << "Precio: ";
cin >> precio;

double total = cantidad * precio;
cout << "Total: " << total;
```

Nivel 3 — Máquina industrial

```
string codigo;
double temperatura;
int piezas;
int respuesta;

cout << "Codigo: "; cin >> codigo;
cout << "Temperatura: "; cin >> temperatura;
cout << "Piezas: "; cin >> piezas;
cout << "Activa? 1=si, 0=no: "; cin >> respuesta;

bool activa = respuesta == 1;
cout << "Reporte: " << codigo << ", "
<< temperatura << " C, " << piezas
<< " piezas, activa=" << activa;
```

El teclado simula datos que en una aplicación industrial podrían provenir de un operador, un PLC o sensores. La estructura es la misma: capturar, almacenar y procesar.

Nivel 4 — Texto completo y datos numéricos

```
string cliente;
string descripcion;
int cantidad;

cout << "Cliente: ";
getline(cin, cliente);

cout << "Descripcion: ";
getline(cin, descripcion);

cout << "Cantidad: ";
cin >> cantidad;
```

Cuando se mezclan lecturas, planifica el orden y limpia el salto de línea si una lectura numérica precede a getline.

Diagramas y representaciones visuales

Inicio → Mostrar solicitud → Usuario escribe → cin/getline recibe → Variable almacena → Validar → Procesar → Mostrar resultado → **Fin**

cin: teclado → flujo → conversión al tipo → variable.

getline: teclado → caracteres hasta Enter → string.

Errores de confusión comunes

- cout muestra; cin recibe.
- cin >> normalmente lee un dato separado; getline lee una línea.
- La variable almacena; la entrada proporciona.
- Enter termina la línea, pero puede quedar pendiente.
- Capturar no es validar.

Errores comunes al trabajar con Entradas de Usuario

1. No incluir iostream

cin y cout no estarán disponibles. Incluye #include .

2. No mostrar una solicitud clara

El usuario puede no saber qué escribir. Utiliza mensajes específicos.

3. Usar cin para una frase

Solo se captura la primera palabra. Usa getline para texto con espacios.

4. Mezclar cin y getline incorrectamente

El salto pendiente puede generar una cadena vacía. Usa ignore en el punto adecuado.

5. No validar

Valores negativos, opciones inválidas o texto inesperado pueden producir resultados incorrectos.

6. Ignorar cin.fail()

El flujo puede quedar en error y las siguientes lecturas no funcionar como esperas.

Buenas prácticas

- Solicita un dato a la vez.
- Usa mensajes claros y consistentes.
- Elige cin para datos simples y getline para frases.
- Valida rangos, opciones y estados de lectura.
- Inicializa variables antes de utilizarlas.
- Separa captura, validación y procesamiento en funciones cuando el programa crezca.
- Prueba entradas correctas, vacías, negativas y con texto inesperado.
- No confíes en que el usuario siempre escribirá correctamente.

Conectando conocimientos

Las variables almacenan los datos; cin y getline los reciben; los operadores los procesan; if valida; los ciclos repiten solicitudes; las funciones organizan la captura; y las clases pueden representar clientes, máquinas, pedidos o formularios.

Ejercicios prácticos

Ejercicio 1 — Comprensión básica

Solicita un nombre y muestra un saludo. Usa el tipo adecuado y decide si necesitas getline. Pista: considera los espacios.

Requisitos: mensajes claros, tipos correctos y pruebas de varios casos.

Pistas: planifica el orden de lectura; usa getline para frases; revisa el salto de línea; considera cin.fail().

Ejercicio 2 — Aplicación

Registra producto, cantidad y precio de un inventario y calcula el total. No uses arreglos ni funciones propias.

Requisitos: mensajes claros, tipos correctos y pruebas de varios casos.

Pistas: planifica el orden de lectura; usa getline para frases; revisa el salto de línea; considera cin.fail().

Ejercicio 3 — Integración

Solicita producción inicial y producción del turno; calcula el total usando una variable de resultado diferente.

Requisitos: mensajes claros, tipos correctos y pruebas de varios casos.

Pistas: planifica el orden de lectura; usa getline para frases; revisa el salto de línea; considera cin.fail().

Ejercicio 4 — Caso práctico

Captura código de máquina, temperatura, velocidad y estado activo. Muestra un reporte y valida que la temperatura no sea negativa.

Requisitos: mensajes claros, tipos correctos y pruebas de varios casos.

Pistas: planifica el orden de lectura; usa getline para frases; revisa el salto de línea; considera cin.fail().

Ejercicio 5 — Reto

Crea un registro de pedido con nombre completo, descripción, cantidad y precio. Valida cantidades positivas, precios mayores que cero y entradas numéricas inválidas.

Requisitos: mensajes claros, tipos correctos y pruebas de varios casos.

Pistas: planifica el orden de lectura; usa getline para frases; revisa el salto de línea; considera cin.fail().

Mini proyecto del capítulo

Nombre: Registro interactivo de una estación de producción

Una empresa necesita una aplicación de consola para que un operador registre el estado de una máquina al comenzar un turno.

Objetivo

Solicitar datos, almacenarlos, validarlos y mostrar un reporte.

Requisitos funcionales

- Código de máquina y nombre completo del operador.
- Temperatura, velocidad y piezas producidas.
- Estado activo/inactivo.
- Validación de cantidades y temperatura.
- Reporte final organizado.

Requisitos técnicos

- Usar string, int, double y bool.
- Usar cin y getline correctamente.
- Usar const para una meta o límite.
- Usar al menos una validación con if.

Etapas

Paso 1: lista los datos y elige tipos.

Paso 2: implementa lecturas simples.

Paso 3: agrega texto completo y limpia el flujo.

Paso final: integra validaciones, reporte y pruebas con datos incorrectos.

Comprueba tus conocimientos

1. ¿Qué es una entrada de usuario?
2. ¿Para qué sirve cin?
3. ¿Qué diferencia hay entre cin >> y getline?
4. ¿Qué biblioteca se necesita?
5. ¿Qué ocurre si se escribe texto para un int?
6. ¿Qué problema aparece al mezclar cin y getline?
7. ¿Para qué sirve cin.ignore()?
8. ¿Qué significa validar?
9. ¿Para qué sirve cin.fail()?
10. ¿Por qué una frase requiere getline?
11. ¿Qué tipos usarías para nombre, temperatura, piezas y estado?

12. ¿Por qué separar captura, validación y procesamiento?
13. ¿Qué diferencia hay entre recibir y procesar?
14. ¿Cómo se relacionan las entradas con las variables?
15. Describe el flujo completo de un programa interactivo.

Respuestas y explicaciones

1. Información proporcionada durante la ejecución.
2. Recibe datos desde la entrada estándar.
3. `cin >>` suele detenerse en espacios; `getline` lee toda la línea.
4. `iostream`.
5. La extracción falla y `cin` entra en estado de error.
6. `getline` puede leer el Enter pendiente y obtener una cadena vacía.
7. Descarta caracteres pendientes.
8. Comprobar que el dato cumple reglas.
9. Detectar fallos de extracción.
10. Porque puede contener espacios.
11. `string`, `double`, `int` y `bool`.
12. Facilita mantenimiento, pruebas y reutilización.
13. Recibir almacena; procesar calcula o decide.
14. Las entradas llenan las variables.
15. Solicitar, leer, almacenar, validar, procesar y mostrar.

Resumen del capítulo

Concepto	Uso	Error común
<code>cin</code>	Leer datos simples	No validar
<code>getline</code>	Leer frases	Olvidar Enter pendiente
<code>cin.ignore</code>	Limpiar entrada	Usarlo sin comprender
<code>cin.fail</code>	Detectar fallos	Continuar tras un error
Validación	Comprobar reglas	Aceptar valores inválidos

Ideas clave que debes recordar

- Las entradas hacen interactivo un programa.
- `cin` recibe datos; `getline` lee líneas completas.
- Leer no significa validar.
- `cin.fail` detecta fallos.
- `ignore` ayuda a limpiar el flujo.

- Los mensajes claros reducen errores.
- Las entradas llenan variables y alimentan cálculos y decisiones.

Mapa de aprendizaje

Variables → **cin/getline** → **captura básica** → **validación** → **procesamiento** → **funciones** → **menús y sistemas reales**.

Este capítulo prepara el siguiente paso: utilizar operadores y expresiones para transformar los datos capturados en cálculos, comparaciones y decisiones.